FOUND: An Open-Source Suite of Horizon-Based Navigation Software

Josh Lando
University of Washington
Guggenheim 211 University of Washington, Seattle WA 98195
jlando@uw.edu

Faculty Advisor: Alvar Saenz-Otero
University of Washington
alvarso@uw.edu

ABSTRACT

FOUND is an open-source software for horizon-based optical navigation. FOUND implements several previously proposed horizon-based optical navigation algorithms that operate within computational constraints of the CubeSat platform. Like other horizon trackers, FOUND uses the horizon of a planetary body to estimate the spacecraft's range and bearing. Although the concept of using horizon-based navigation dates back to the Apollo missions, no system has ever been built for the CubeSat platform. The University of Washington's Husky Satellite Lab plan to test FOUND on orbit in 2028 as part of a larger initiative to enable interplanetary CubeSat missions.

INTRODUCTION

Position determination is a key requirement for most satellite missions. The default for position determination is the Global Navigation Satellite System (GNSS). GNSS offers many benefits: sub-meter level positioning, full orbit coverage, and precise timing. Unfortunately, there are three critical weaknesses for deep space and military applications: (1) accuracy degrades quickly with increasing distance from Earth, (2) dependency on GNSS constellations increases mission risk, (3) GNSS codes are susceptible to jamming. A solution that addresses these weaknesses is horizon-based optical navigation (OpNav), which uses an image of a body-space boundary, called the limb, to compute the inertial state of the satellite.

The need for deep space, GNSS independent navigation has never been greater. Interest from both the government and private companies in a sustained lunar presence means that reliable, self-contained, lunar-navigation capabilities will be essential. These navigation systems are especially needed for crewed missions, where a loss in communications cannot lead to a loss in vehicle state knowledge.

To address this and prepare for the University of Washington's Husky Satellite Lab's very own lunar mission, we present FOUND: Open-Source Universal Navigation Determiner. FOUND is an open-source C++ project designed to run within the constraints of the CubeSat platform. FOUND operates as the companion software for LOST: Open-Source Star Tracker [3]. Together, LOST and FOUND make up Husky Satellite's Lab suite of OpNav software capable of computing the inertial state of a satellite.

In this paper we discuss:

1. Software Architecture
2. FOUND Algorithms
3. Testing & Methodologies
4. Results & Discussion
5. Appendix: Coordinate Transformations & Code

BACKGROUND

History

Despite horizon-based OpNav's lack of notoriety, it has a storied history with the first implementation seen on the Apollo missions as a backup system in case of loss of communications. Later, horizon-based OpNav reappeared as a primary navigation method for the NASA Voyager program [4] [5]. Only recently

has the first demonstration of a fully autonomous system emerged as part of the Artemis navigation system [2].

Camera Coordinates

We define four coordinate systems:

1. World Coordinate System ($\mathbb{R}^3$): The Earth-Centered, Earth-Fixed (ECEF) coordinates expressed in the J2000 reference frame
2. Camera Coordinate System ($\mathbb{R}^3$): The coordinate system relative to FOUND's camera
    - The z-axis points along the optical axis, with the other axes forming a plane parallel to the camera face
3. Image Coordinate System ($\mathbb{P}^2$): The plane sharing the same x and y axis as the camera coordinate system, located a unit distance from the optical center of the camera
    - points in $\mathbb{P}^2$ can be written in homogeneous coordinates, $[x, y, 1]^T \in \mathbb{P}^2$
4. Pixel Coordinate System ($\mathbb{R}^2$): The coordinates used by a computers graphical display
    - Equivalent to the index used to access pixel information

These frames will be referred to as $\mathbf{W}$, $\mathbf{C}$, $\bar{\mathbf{X}}$, and $\mathbf{U}$.

The transformation from camera to image coordinates using the pinhole camera model, and the transformation between the image and pixel coordinates using the camera calibration matrix are detailed in the Appendix.

SOFTWARE ARCHITECTURE

Within FOUND, navigation is straightforward. 4 algorithms completed sequentially takes an image and attitude knowledge to an inference of position:

1. Edge Detection: Takes a monochrome image I and returns the set of edge coordinates corresponding to the celestial limb, $\{\mathbf{u}_i\}_{i=1}^n \in \mathbf{U}$
2. Edge Filtering: Refines $\{\mathbf{u}_i\}_{i=1}^n$
3. Distance Determination: Uses the edge estimates, $\{\mathbf{u}_i\}_{i=1}^n$, and camera's attitude to compute the position of the celestial body with respect to the camera origin, $\mathbf{r_C}$ (the subscript denotes, $\mathbf{r} \in \mathbf{C}$)
4. Coordinate Transformation: Uses the camera's attitude to compute the position of the satellite in world coordinates, $\mathbf{r_W}$, from $\mathbf{r_C}$

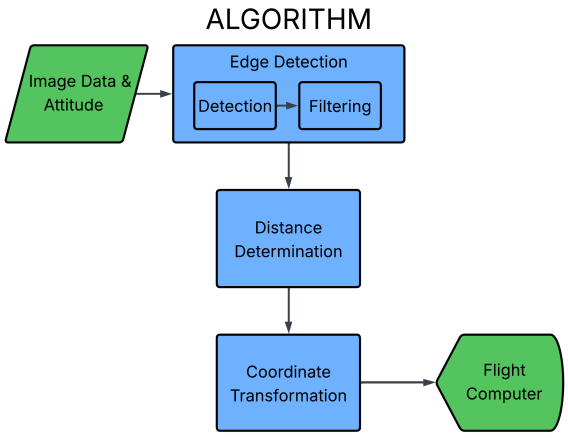


Figure 1: System architecture diagram

The attitude of the satellite is obtained by an independent attitude inference system.

ALGORITHM OVERVIEW

At a high level, the specific implementations for the architecture above is as follows:

1. Edge Detection: Canny (Sobel) filtering with non-maximum suppression along the sunlight direction
2. Edge Filtering: Zernike Moment-based subpixel correction
3. Distance Determination: The Christian-Robinson Algorithm (CRA)
4. Coordinate Transformation: Rotation and flip from $\mathbf{C}$ to $\mathbf{W}$

In this paper, we will explain the CRA since it is specific to horizon-based navigation. The edge detection stage is composed of common computer vision

algorithms. We will leave the explanation of these algorithms to Ref. [2].

Christian-Robinson Algorithm [1][2]

The CRA is a non-iterative, least squares solution for the position of a triaxial ellipsoidal body relative to the camera. The main idea of the CRA is that it transforms any triaxial ellipsoidal body into a space where it is a unit sphere, making the distance determination problem trivial.

Suppose we have n horizon points in the image, $\{\mathbf{u}_i\}_{i=1}^n$. We wish to transform these points into $\bar{\mathbf{X}}$, but first, we must rewrite $\{\mathbf{u}_i\}_{i=1}^n$ in homogeneous coordinates, $\bar{\mathbf{u}}_i = [u, v, 1]^T$. We now project the points into the camera coordinate system via:

$$\bar{\mathbf{x}}_i = K^{-1}\bar{\mathbf{u}}_i \tag{1}$$

where K^{-1} is the camera projection matrix. Next, we use the following transformation to take the homogeneous image coordinates, $\{\bar{\mathbf{x}}_i\}_{i=1}^n$, into a modified frame where a triaxial ellipsoid becomes the unit sphere,

$$\mathbf{s}_i = (DT_P^C K^{-1})\bar{\mathbf{x}}_i. \tag{2}$$

Here D is one over the principal axes of the triaxial ellipsoid, $D = \text{diag}[\frac{1}{A}, \frac{1}{B}, \frac{1}{C}]$, and T_P^C is the rotation from camera to planetary coordinates. T_P^C can be computed from an on board star tracker or other attitude determination device.

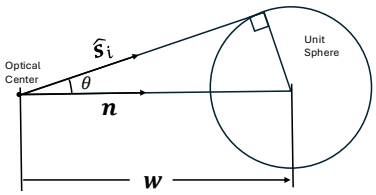


Figure 2: Illustration of unit sphere geometry

We can now take advantage of the simple geometry of the unit sphere to find $\mathbf{n}$, the vector from the camera origin to the center of the unit sphere in our transformed frame. Consider the figure above. First, we compute the unit vectors of $\mathbf{s}_i$ denoting them $\hat{\mathbf{s}}_i = \mathbf{s}_i/\|\mathbf{s}_i\|$. Next, note that $\mathbf{n}$, $\hat{\mathbf{s}}_i$, and the unit radius form a right triangle (see Figure 2). Since $\mathbf{n}$ and the radius are constant for all $i \in \{1, \ldots, n\}$, θ, the angle between $\mathbf{s}_i'$ and $\mathbf{n}$ is also constant. Then, taking the dot product between $\mathbf{n}$ and $\mathbf{s}_i$ we find:

$$\mathbf{n}^T\hat{\mathbf{s}}_i = \|\mathbf{n}\|\cos\theta = \text{const}$$

Therefore we may write,

$$\begin{bmatrix}\hat{\mathbf{s}}_\mathbf{1}^\mathbf{T}\\ \vdots \\ \hat{\mathbf{s}}_\mathbf{n}^\mathbf{T}\end{bmatrix}\mathbf{n} = \begin{bmatrix}1\\ \vdots \\ 1\end{bmatrix} \tag{3}$$

We can now use the least squares solution of n to compute w. To find the length of $\mathbf{w}$ we use $\|\mathbf{n}\| = 1/\cos\theta$ to rewrite $\sin\theta = 1/\|\mathbf{w}\|$ as,

$$\begin{aligned}\|\mathbf{w}\| &= \sqrt{\frac{1}{\sin^2\theta}}\\ &= \sqrt{\frac{1}{1-\cos^2\theta}}\\ &= \|\mathbf{n}\|(\mathbf{n}^T\mathbf{n}-1)^{-\frac{1}{2}}\end{aligned} \tag{4}$$

We then compute $\mathbf{w}$ by scaling the unit vector $\hat{\mathbf{n}}$ by the result of equation (4). Finally transforming $\mathbf{w}$ back into our camera frame, C, we find that the position of the celestial body is,

$$\mathbf{r}_C = (\mathbf{n}^T\mathbf{n}-1)^{-\frac{1}{2}}T_C^P D^{-1}\mathbf{n} \tag{5}$$

where T_C^P is the inverse T_P^C, $T_C^P = (T_P^C)^T$, making it the rotation from planetary to camera coordinates.

In summary, we have an algorithm to go from the set of horizon points in our image to the location of the celestial body with respect to the camera.

TESTING METHODOLOGIES

FOUND is evaluated at both a modular and end-to-end level. For the simulations, Earth is modeled as a triaxial ellipsoid with a polar-radius of 6356.75 km and an equatorial-radius of 6378.14 km. Altitudes range from low earth orbit to halfway between the Earth and the Moon.

The Image Generator produces a monochrome image, I, and the Limb Generator produces edge points,

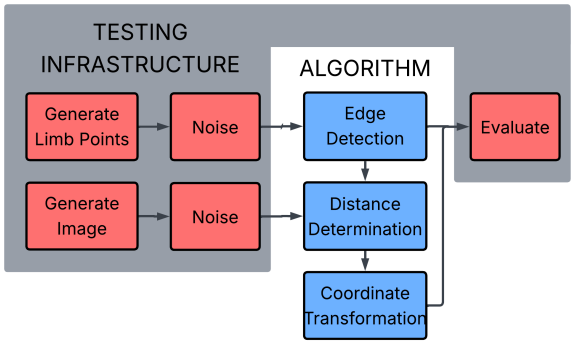


Figure 3: Testing infrastructure diagram

$\{\mathbf{u}_i\}_{i=1}^n$. The analytical solution of the limb's pixel conic derived in Ref. [2] was used to generate synthetic data.

Statistics

After each run the true position, $\mathbf{t}$, and the estimated position, $\mathbf{r}$, are recorded. Performance-benchmarks track 3 statistics:

1. Centroid Error: measures the distance between the estimated and true satellite to Earth vector in pixel coordinates.

$$\Delta c = \left\| K \left(\begin{bmatrix} \mathbf{t}_1/\mathbf{t}_3 \\ \mathbf{t}_2/\mathbf{t}_3 \\ 1 \end{bmatrix} - \begin{bmatrix} \mathbf{r}_1/\mathbf{r}_3 \\ \mathbf{r}_2/\mathbf{r}_3 \\ 1 \end{bmatrix} \right) \right\| \tag{6}$$

   where K is the camera calibration matrix.

   Both the true and estimated position are first projected to image coordinates and then transformed to homogeneous pixel coordinates. The distance between pixel coordinates is the centroid error.

2. Apparent Radius Error: measures the range accuracy in pixel coordinates.

$$\Delta r = \frac{R_P \mu}{f} \cdot \frac{\|\mathbf{r}\| - \|\mathbf{t}\|}{\|\mathbf{r}\|\|\mathbf{t}\|} \tag{7}$$

   The apparent radius can be visualized as the radius of the celestial body seen in the image.

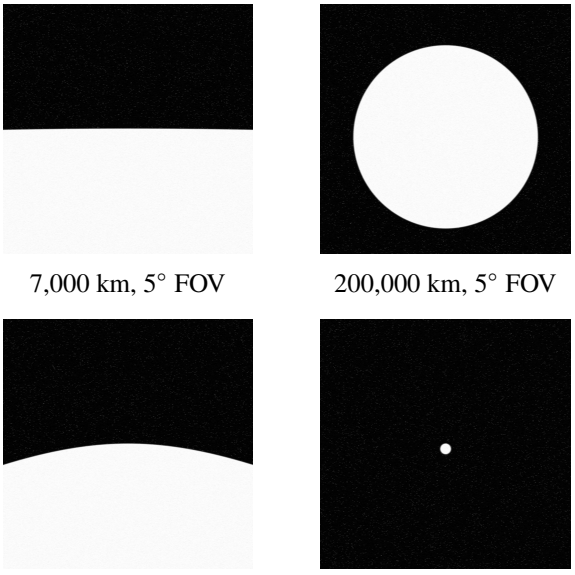
3. Position Error: measures the distance between the estimated and true position in units of meters.

$$\Delta p = \|\mathbf{t} - \mathbf{r}\| \tag{8}$$

Pixels are a unitless measure that provide a direct connection to the theoretical limit of the system: pixel discretizations. FOUND achieves subpixel accuracy through Zernike moment updates and a least squares solution to the spherical OpNav problem.

Camera Hardware Evaluation

FOUND's performance depends on camera hardware and the distance from the celestial body. A narrow FOV lens results in a zoomed in image, capturing a straight line at close ranges and the full Earth at far ranges. On the other hand, a wide FOV lens results in a zoomed out image capturing the curvature of the Earth's limb at close ranges. At far ranges, a wide FOV lens images Earth as a tiny circle, containing minimal limb information.



7,000 km, 5° FOV | 200,000 km, 5° FOV

7,000 km, 80° FOV | 200,000 km, 80° FOV

Figure 4: Effect of FOV on Earth's size in the image

12 camera setups were tested:

- Fields of view: 5°, 20°, 50°, 85°
- Pixel resolutions: 512×512, 1024×1024, 2048×2048

For each camera setup, we simulated 250 satellite states at 20 ranges, resulting in 60,000 total data points. The simulation was conducted as follows:

1. Generate Limb Points & Attitude: $\{\mathbf{u}_i\}_{i=1}^n, T_P^C$

- A maximum of 1 limb point per pixel
- limb points are truncated to a single decimal digit
- $\{\mathbf{u}_i\}_{i=1}^n$ perturbed with 0.2σ Gaussian noise

2. Distance Determination & Coordinate Transformation: $\{\mathbf{u}_i\}_{i=1}^n, T_P^C \rightarrow \mathbf{r}_W$
3. Statistics: $\mathbf{r}_W \rightarrow \Delta c, \Delta r, \Delta p$

Regression Evaluation

Figure 5 shows the output of the edge detection stage for a noisy image. The false edge points seen in the figure cause the CRA to diverge. This can be handled by implementing an additional edge filtering algorithm such as blob detection or by choosing a more robust regression algorithm. In this paper, we will explore the second option.

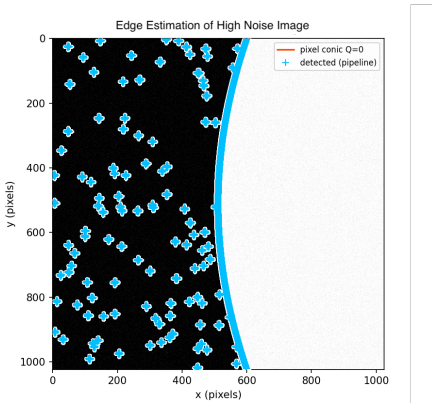


Figure 5: Edge detection in high noise environment

Two different regression algorithms were tested in a high noise environment:

- Total Least Squares (TLS): baseline suggested in Ref. [1, 2]
- Random Sample Consensus (RANSAC): robust alternative capable of outlier rejection

For both TLS and RANSAC we simulated 250 satellite states at 20 ranges, resulting in 10,000 total data points. The simulation was conducted using a camera with an FOV of 50° and 2048×2048 pixel resolution. The simulation was conducted as follows:

1. Generate Limb Points & Attitude: $\{\mathbf{u}_i\}_{i=1}^n, T_P^C$
   - A maximum of 1 limb point per pixel
   - limb points are truncated to a single decimal digit
   - $\{\mathbf{u}_i\}_{i=1}^n$ perturbed with 0.4σ Gaussian noise
   - added 20 random edge points to $\{\mathbf{u}_i\}_{i=1}^n$
2. Distance Determination & Coordinate Transformation: $\{\mathbf{u}_i\}_{i=1}^n, T_P^C \rightarrow \mathbf{r}_W$
3. Statistics: $\mathbf{r}_W \rightarrow \Delta c, \Delta r, \Delta p$

Full Pipeline Evaluation

We simulated 250 satellite states at 20 ranges, resulting in 5,000 total data points. The simulation was conducted using a camera with an FOV of 50° and 2048×2048 pixel resolution. The simulation was conducted as follows:

1. Generate Image & Attitude: I, T_P^C
   - T_P^C perturbed with 10 arcsec standard deviation Gaussian noise
2. Edge Detection, Distance Determination, & Coordinate Transformation: $I \rightarrow \mathbf{r}_W$
3. Statistics: $\mathbf{r}_W \rightarrow \Delta c, \Delta r, \Delta p$

RESULTS

We find it useful to plot the statistics' availability when making comparisons and the statistics' bias and spread when evaluating individual performance.

Camera Hardware Evaluation

Figure 6 plots the centroid availability for different camera setups, $\Delta c < 0.05$. At close ranges, all camera setups struggle to converge on an accurate centroid. As the satellite moves further away from Earth, the centroid accuracy increases with wide FOVs converging quicker than the narrow FOVs. As range increases past the optimal altitude, Earth approaches a point in the image causing the centroid accuracy to degrade.

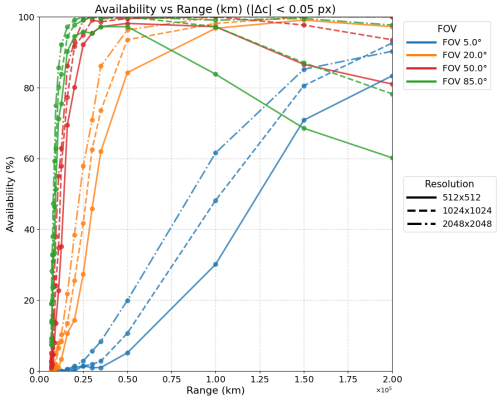


Figure 6: Centroid error for different camera setups

In figure 7, we observe increasing availability followed by a drop-off similar to figure 6. Availability for apparent radius is consistently greater than availability for the centroid, implying that FOUND has better range accuracy than centroid accuracy.

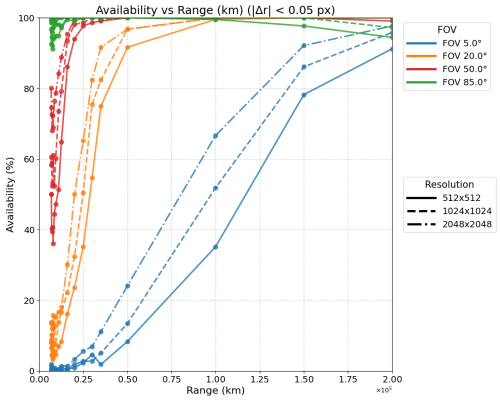


Figure 7: Apparent radius error for different camera setups

In figure 8, a best fit quadratic was fitted to the percentage of measurements with $\Delta p < 1$ km. The first thing to note is that quadratic lines corresponding to the same color (FOV) achieve their maximums at similar ranges irrespective of pixel resolutions. This supports the conclusion that there is an optimal operating range for every lens setup. We find that this optimal range occurs at the minimum distance the camera is able to fully capture Earth's limb.

The dashed lines corresponding to larger pixel resolution consistently perform better than the solid lines corresponding to lower pixel resolutions. This shows how FOUND's accuracy can be improved by using a higher resolution camera, sacrificing SWaP.

The narrow FOV cameras max out at a lower availability than the wide FOV cameras. The reason for this becomes clear when error is thought of as a percentage of the range, $\%\text{error} = 100 \cdot \Delta p / \|\mathbf{t}\|$. At 200,000 kilometers, the percentage error for 1 km distance error is 2 orders of magnitude more accurate than at 7,000 kilometers.

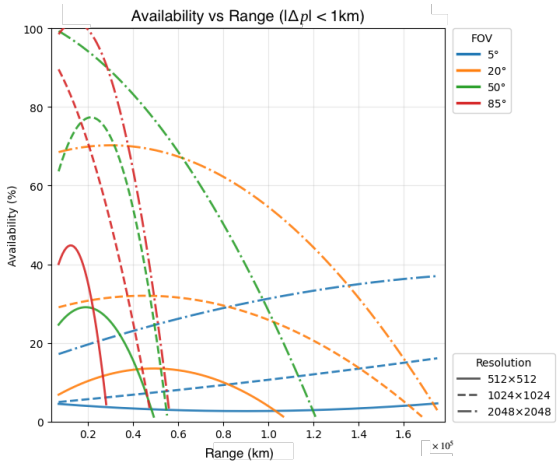


Figure 8: Position error for different camera setups

Regression Evaluation

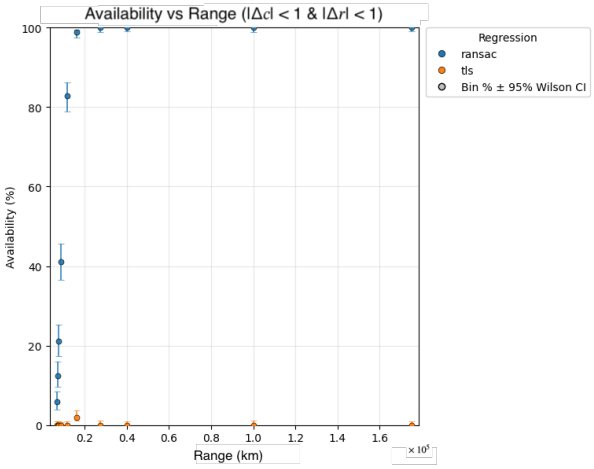


Figure 9: Regression performance

Figure 9 plots the availability of TLS and RANSAC regression in a high noise environment. In this setup, TLS regression always fails to converge. On the other hand, RANSAC regression achieves nearly 100% availability once the entirety of the limb becomes

visible in the image. When the limb is only partially visible, RANSAC regression produces mixed results.

Based on this evaluation, additional edge filtering is recommended for missions that do not capture the full geometry of the celestial body. For missions that are able to capture the full geometry of the celestial body, RANSAC regression is a feasible solution to handle false edge points.

It is possible that with the right camera and intensity threshold stars might not be detected as limb points. However, it is absolutely necessary to handle edge points caused by dead pixels.

Full Pipeline Evaluation

In Figure 10, FOUND struggles to converge to a subpixel centroid estimate when the full limb of Earth is not visible, demonstrating a large bias at short ranges.

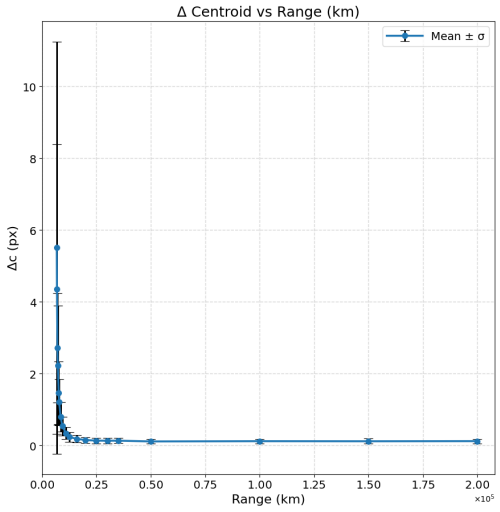


Figure 10: Centroid accuracy for full pipeline simulation

In Figure 11, FOUND accurately predicts the apparent radius at close ranges, but exhibits a positive bias as range increases, causing an underestimate in range. This implies that the edge detection stage is detecting the edge points on the space side of the limb boundary. Indeed, in Figure 12 we see that this is true.

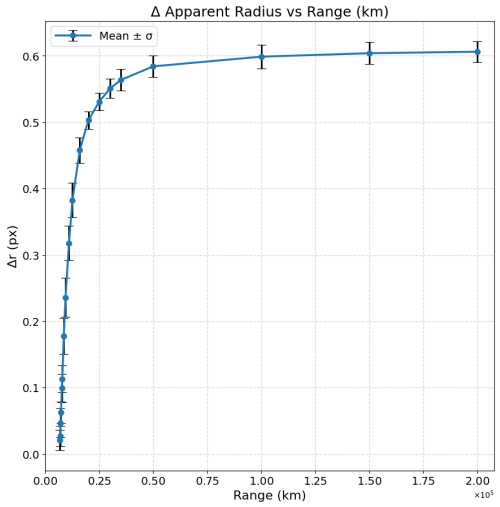


Figure 11: Apparent radius accuracy for full pipeline simulation

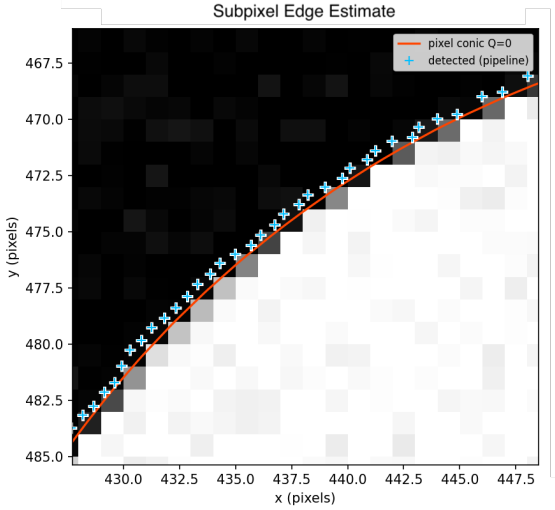


Figure 12: Overestimation of subpixel edge points

Figure 13 plots the position error of FOUND for the full pipeline simulation. At altitudes below 10,000 km the error remains below 3,000 km 99% of the time, but as range increases, the position error's spread and bias explode.

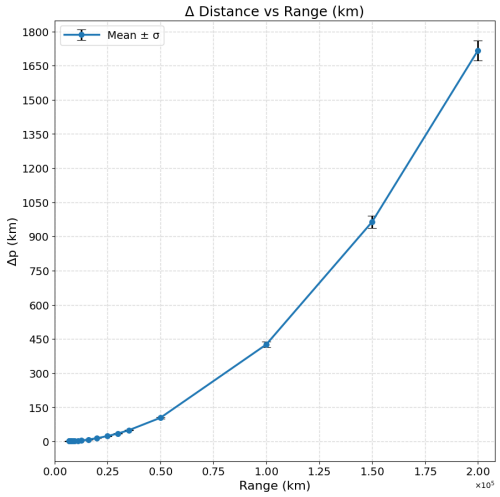


Figure 13: Position accuracy for full pipeline simulation

Relating Equation (7) to (8) we find that position error grows like the distance squared:

$$\Delta p \geq \frac{f}{R_P \mu} \Delta r \|\mathbf{r}\| \|\mathbf{t}\|.$$

In comparison, the error caused by attitude and centroid uncertainty is negligible:

$$\Delta p > \left[2 \sin\left(\frac{\Delta\theta}{2}\right) + \frac{\mu}{f} \Delta c \right] \|\mathbf{r}\|.$$

Therefore, it is imperative to decrease the apparent radius error.

The full pipeline simulation exhibited major biases in the edge detection stage. These biases could have been mitigated by tuning the edge detection stage's hyper-parameters beforehand. However, the hyper-parameters would then be optimized for our image generator instead of the real world. This emphasizes the need for a real-world dataset to pre-tune FOUND's hyper-parameters before launch.

CONCLUSION

In this paper, we have presented the first open source implementation of horizon-based navigation and confirmed that it solves for the inertial state of the satellite with sub pixel accuracy. We discussed methods to select the correct focal length and pixel resolution based on mission parameters and evaluated a method for mitigating the impacts of false edge points.

This marks the completion of the University of Washington's LOST & FOUND initiative to create a fully open-source software solution capable of solving for the inertial state of a satellite anywhere in our solar system.

LIMITATIONS & FUTURE WORK

FOUND is a starting point for future horizon-based OpNav development, allowing researchers to focus on improvements instead of rewriting auxiliary pipeline and testing infrastructure.

We see the edge detection stage as the main area in need of improvement. Specifically, atmospheric blur is an understudied weakness of horizon-based OpNav. The current solution, an offset to the celestial body's principle axis, treats atmospheric scattering as a constant effect, ignoring the sun angle dependency of Rayleigh scattering. A main blocker for algorithm development in this field is the lack of real world data. Despite Earth being the most widely studied celestial body, images of the horizon to space transition with accurate position and attitude data are not widely available. We hope to remedy this lack of data with the launch of HuskySat2 in 2028.

In the future we plan on implementing the the covariance algorithm suggested on pg. 599 Ref. [2] as well as a renderer capable of accurately replicating atmospheric effects.

Finally, it is important to note that FOUND is far from being fit for embedded applications. At the time of writing, FOUND relies heavily on heap memory to store edge information. In the future, we plan to add a variant of the core algorithms that use stack memory only, which will allow us to provide a guaranteed maximum memory footprint based on the size of the image used. Additionally, porting the edge detection stage to an embedded system often requires hardware specific optimization for efficient run times. However, for missions willing to take the performance hit that comes with running FOUND on a Unix-based system, FOUND is space ready.

ACKNOWLEDGMENTS

I would first like to thank Tri Nguyen who started the FOUND project and has continued to give invaluable advice after graduation. Additionally, thank you to Senuka Liyanage for writing the CRA, Eesha Jain for writing the subpixel edge detection algorithm, Ria Agarwal for writing the integration testing scripts, and the rest of the FOUND team for work that is beyond the scope of this paper. A final thanks to the University of Washington's Husky Satellite Lab and Faculty Advisor, Alvar Saenz-Otero for supporting this project.

REFERENCES

[1] John A. Christian. A tutorial on horizon-based optical navigation and attitude determination with space imaging systems. *IEEE Access*, 9:19819–19853, 2021.

[2] John A. Christian. *Fundamentals of Spacecraft Optical Navigation*. John Wiley & Sons, Incorporated, 1 edition, 2025.

[3] Mark Polyakov, Edward Zhang, and Karen Haining. LOST: An open-source suite of star tracking software. In *Proceedings of the 37th Annual Small Satellite Conference*, Frank J. Redd Student Competition, Utah State University, Logan, UT, August 2023.

[4] Joseph E. Riedel, William M. Owen, J. A. Stuve, Stephen P. Synnott, and Robin M. Vaughan. Optical navigation during the Voyager Neptune encounter. In *AIAA/AAS Astrodynamics Conference*, 1990. AIAA Paper 90-2877.

[5] Stephen P. Synnott, A. J. Donegan, Joseph E. Riedel, and J. A. Stuve. Interplanetary optical navigation: Voyager Uranus encounter. In *AIAA/AAS Astrodynamics Conference*, 1986. AIAA Paper 86-2113.

APPENDIX

Camera → Image

A point in camera coordinates $\mathbf{s} \in \mathbb{R}^3$ can be mapped to a point in image coordinates, $\bar{\mathbf{x}}$, using the pinhole camera model,

$$\mathbb{F} : C \to \bar{\mathbf{X}}, \quad \mathbf{s} \mapsto \begin{bmatrix} \mathbf{s}_1/\mathbf{s}_3 \\ \mathbf{s}_2/\mathbf{s}_3 \\ 1 \end{bmatrix} \tag{9}$$

Image → Pixel

A point in image coordinates $\bar{\mathbf{x}} \in \mathbb{P}^2$ can be mapped to homogeneous pixel coordinates $\bar{\mathbf{u}}$ via the camera calibration matrix:

$$\bar{\mathbf{u}} = K\bar{\mathbf{x}} \tag{10}$$

The camera calibration matrix is:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{11}$$

where:

- f_x — focal length in the x-direction (pixels)
- f_y — focal length in the y-direction (pixels)
- c_x — principal point x-coordinate (pixels)
- c_y — principal point y-coordinate (pixels)

The pixel focal lengths are related to the physical focal length f and pixel size p by $f_x = f/p_x$ and $f_y = f/p_y$. The principal point (c_x, c_y) is typically located near the image center and represents where the optical axis intersects the image plane.

The camera calibration matrix K is full rank and has a well defined inverse that takes pixel coordinates to image coordinates.

Code

The FOUND repository and further documentation can be found at `https://github.com/UWCubeSat/found`.

The synthetic data generation tools can be imported from PyPi. The repository and further documentation can be found `https://github.com/UWCubeSat/found-tools`.

The simulation scripts and further documentation can be found at `https://github.com/UWCubeSat/found-integration-testing`.